project_summary_and_analysis

May 12, 2026

This project extends Borovykh et al. (2018), *Conditional Time Series Forecasting with Convolutional Neural Networks*, by applying the conditional WaveNet framework to a cross-sectional equity return prediction and portfolio construction setting. Building on the paper's comparison between unconditional and conditional convolutional architectures, this project evaluates whether conditioning on market-level signals improves stock-level return forecasts and trading performance.

An interactive Streamlit dashboard for the project is available here:

Launch Interactive Dashboard

This notebook serves as the main pipeline for the Python-based backtesting framework. It integrates the core modules of the project, including data preparation, feature engineering, model construction, model training and validation, portfolio construction, out-of-sample backtesting, regime analysis, transaction cost modeling, and the Streamlit dashboard.

The workflow is designed to call the underlying Python scripts as reusable modules rather than redefining all functions directly inside the notebook. This keeps the main logic modular and organized, while allowing the notebook to act as a clear execution and experiment interface.

Specifically, this pipeline loads the prepared market and stock-level data, computes the required features and market signals, constructs the uWN, cWN, and LSTM models, runs walk-forward training and validation, generates out-of-sample predictions, builds long-short and long-only portfolios, evaluates performance metrics, and saves the final results and plots for further analysis.

After the baseline backtest is completed, the project adds three additional layers of analysis. First, the regime analysis evaluates whether strategy performance changes across different market environments, such as the pre-2020 bull market, the COVID crash, and the 2022 bear market. Second, the transaction cost modeling module uses the saved prediction files to test how portfolio performance changes under different trading costs and rebalancing frequencies. Third, the Streamlit web app provides an interactive interface for exploring the data, viewing backtest results, analyzing regimes, and running portfolio simulations without retraining the models.

The purpose of this notebook is to provide a centralized and reproducible entry point for running the full modeling, backtesting, robustness, and dashboard workflow.

0.1 0. Project Structure Overview

This project is organized as a modular Python-based research and backtesting framework. The main components are separated into different folders according to their functions, including data processing, feature engineering, model implementation, backtesting, robustness analysis, transaction cost modeling, Streamlit web visualization, and result storage.

All the files should be formed as below modular python framework.

```
IEOR4733-Final-Project/
  data/
     raw/
        ohlcv.parquet             # Raw stock-level OHLCV data
        signals.parquet           # Raw market signal data
        sp500_tickers.csv         # S&P 500 ticker universe

     processed/
        features.parquet          # Processed feature dataset

     data_pipeline.py              # Data loading, cleaning, processing, and saving
     features.py                   # Feature engineering for stock-level and market-level fea

  model/
     uWNcWN2.py                    # Main model architecture for uWN and cWN
     lorenz_test2.py               # Lorenz test and model validation
     backtest_uWNcWN_LSTM2.py      # Training, validation, prediction, portfolio construction

  robustness/
     regime_analysis.py            # Regime-level performance analysis

  transaction_cost/
     transactioncost.py            # Portfolio comparison with transaction cost modeling
     metrics.csv                   # Transaction-cost-adjusted performance metrics
     daily_returns.csv             # Daily strategy returns under transaction costs
     cumulative_returns.csv        # Cumulative strategy returns under transaction costs
     long_short_comparison.png     # Long-short portfolio comparison plot
     long_only_comparison.png      # Long-only portfolio comparison plot

  pages/
     01_data_explorer.py           # Streamlit page for data and feature exploration
     02_backtest_results.py        # Streamlit page for backtest result visualization
     03_regime_analysis.py         # Streamlit page for regime analysis
     04_run_simulation.py          # Streamlit page for interactive portfolio simulation

  results/
     lorenz_test/                  # Outputs from the Lorenz test
     uWNcWN_LSTM2/                 # Backtesting and Regime analy sisoutputs, metrics, predic

  app.py                            # Streamlit dashboard entry point
  Main_Pipeline.ipynb               # Main notebook pipeline that coordinates all Python modul
  README.md                         # Project documentation
  .gitignore                        # Git ignore configuration
  .gitattributes                    # Git attributes configuration
```

The `./data/` folder contains the data-related modules and datasets. The raw market data is stored in `./data/raw/`, while processed datasets can be stored in `./data/processed/`. The script

`./data/data_pipeline.py` is used to prepare the raw data, and `./data/features.py` contains the feature engineering functions for generating stock-level features and market-level signals.

The `./model/` folder contains the core model and evaluation scripts. The file `./model/uWNcWN2.py` define the unconditional WaveNet and conditional WaveNet model structures. The backtesting logic is implemented in `./model/backtest_uWNcWN_LSTM2.py`, while `./model/lorenz_test2.py` is used for additional model evaluation through the Lorenz test.

The `./robustness/` folder contains the regime analysis script. This script uses the daily backtest return files and evaluates strategy performance across different market regimes. The regime outputs can be saved into the selected results folder and displayed in the Streamlit dashboard.

The `./transaction_cost/` folder contains the transaction cost modeling script and its outputs. This module does not retrain the neural network models. Instead, it loads the saved prediction files from the backtest results folder and reconstructs long-short and long-only portfolios under alternative transaction cost and holding period assumptions.

The `./pages/` folder and `app.py` form the Streamlit web application. The web app provides an interactive interface for data exploration, backtest visualization, regime analysis, and simulation of portfolio rules using pre-computed model predictions.

The `./results/` folder is used to store the outputs generated by the pipeline, including Lorenz test results, baseline backtest results, model prediction files, regime analysis outputs, and sensitivity analysis outputs.

This notebook serves as the main execution pipeline of the project. Instead of rewriting all functions inside the notebook, it imports the existing Python modules and coordinates the full workflow from data preparation to model testing, backtesting, robustness analysis, transaction cost modeling, and web app deployment.

0.2 1. Data preparation

In this section, we prepare the data required for model training, validation, and backtesting. Since the project is organized into separate `./data/` and `./model/` folders, we first update the Python search path so that the notebook can import the corresponding modules directly.

The data preparation process is mainly implemented in `./data/data_pipeline.py`. This module is responsible for loading the raw market data, cleaning and restructuring the dataset, and saving the processed files that will be used in the later modeling pipeline.

We also import `./data/features.py`, which contains the feature engineering functions. These functions are used to construct stock-level features and market-level signals, which serve as the input variables for the uWN, cWN, and LSTM models.

After importing the required modules, we run `data_pipeline.run_pipeline()` to execute the full data preparation process. This step ensures that the raw data is transformed into the required format before entering the model construction, validation, and backtesting stages.

[ ]:
```
import sys
import importlib

sys.path.append("./model")
```

```python
sys.path.append("./data")

from data import features
from data import data_pipeline
```

[ ]:
```python
data_pipeline.run_pipeline()
```

```
=======================================================
  Phase 1 - S&P 500 Data Pipeline (extended features)
=======================================================


           Feature Library Summary

   A Returns Momentum                9 features
   B Trend                           8 features
   C Volatility                      5 features
   D Oscillators                     7 features
   E Volume Liquidity                6 features
   F Price Structure                 5 features
   G Market Signals                  8 features

   Total (per-stock)                40 features
   Total (incl. market signals)     48 features


Step 1: Tickers
  [cache] Tickers loaded from disk.

Step 2: OHLCV
  [cache] OHLCV loaded from disk.

Step 3: Market signals
  [cache] Signals loaded from disk.

Step 4: Stock features
  Computing features for 503 tickers …
    … 0/503
    … 100/503
    … 200/503
    … 300/503
    … 400/503
    … 500/503

Step 5: Market-wide signals

Step 6: Merging & cleaning
```

```
Final shape: (1776807, 48)  (48 features)
Saved -> data/processed/features.parquet
```

0.3 2. Reproduce the Model

In this part, we implement the unconditional WaveNet model, uWN, and the conditional WaveNet model, cWN, following the structure described in the paper. The corresponding model code is stored in `./model/uWNcWN2.py`.

Following the paper, we also conduct the Lorenz test to evaluate the model. The implementation of this test is stored in `./model/lorenz_test2.py`.

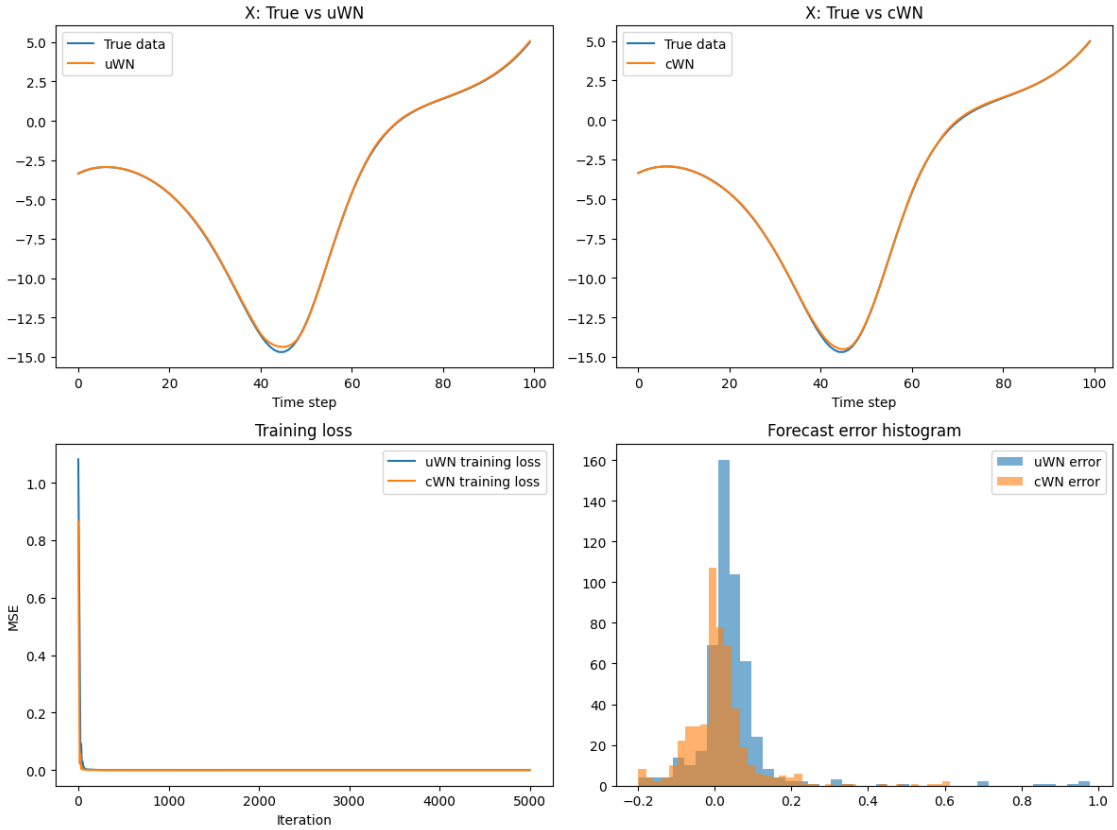
Here, we import the required model and testing modules, reload them to make sure the latest code is used, and then present the results of the Lorenz test.

[ ]:
```
from model import uWNcWN2
from model import lorenz_test2

importlib.reload(uWNcWN2)
importlib.reload(lorenz_test2)
```

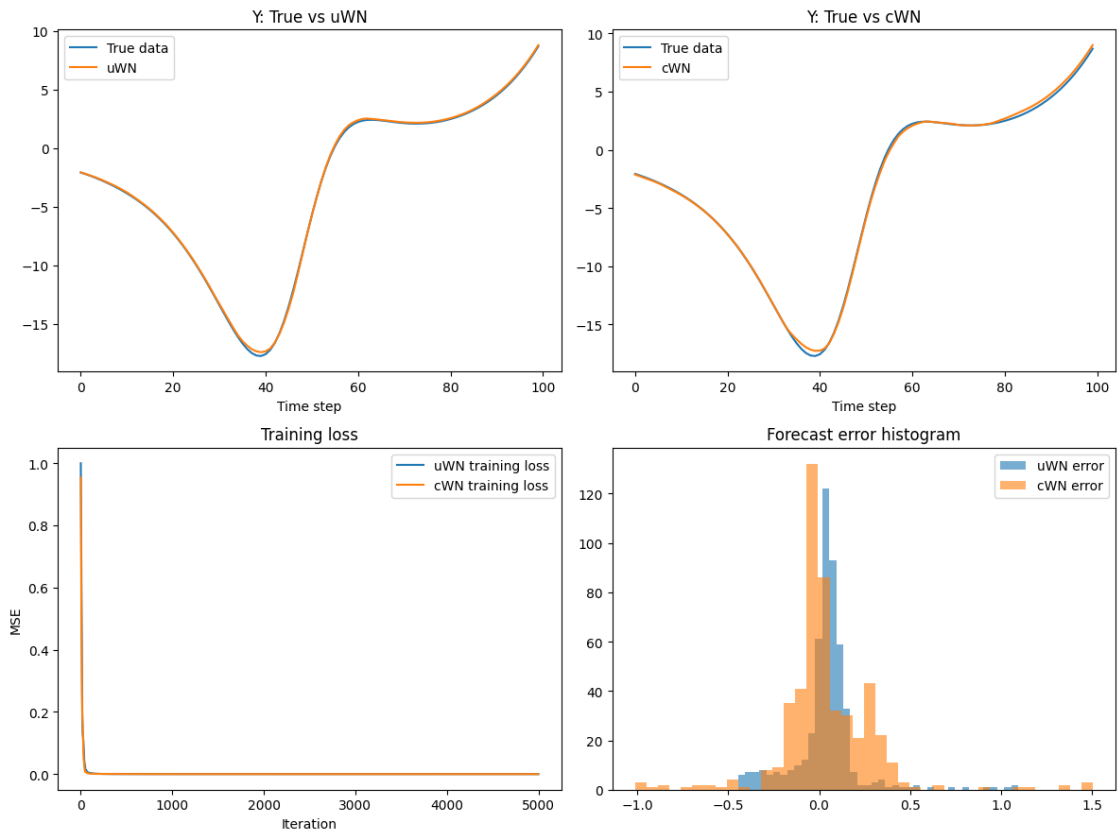
[ ]:
```
<module 'model.lorenz_test2' from
'd:\\Columbia\\AlgoTrading\\FinalProject\\IEOR4733-Final-
Project\\model\\lorenz_test2.py'>
```

[ ]:
```
lorenz_test2.lorenz_test()
```

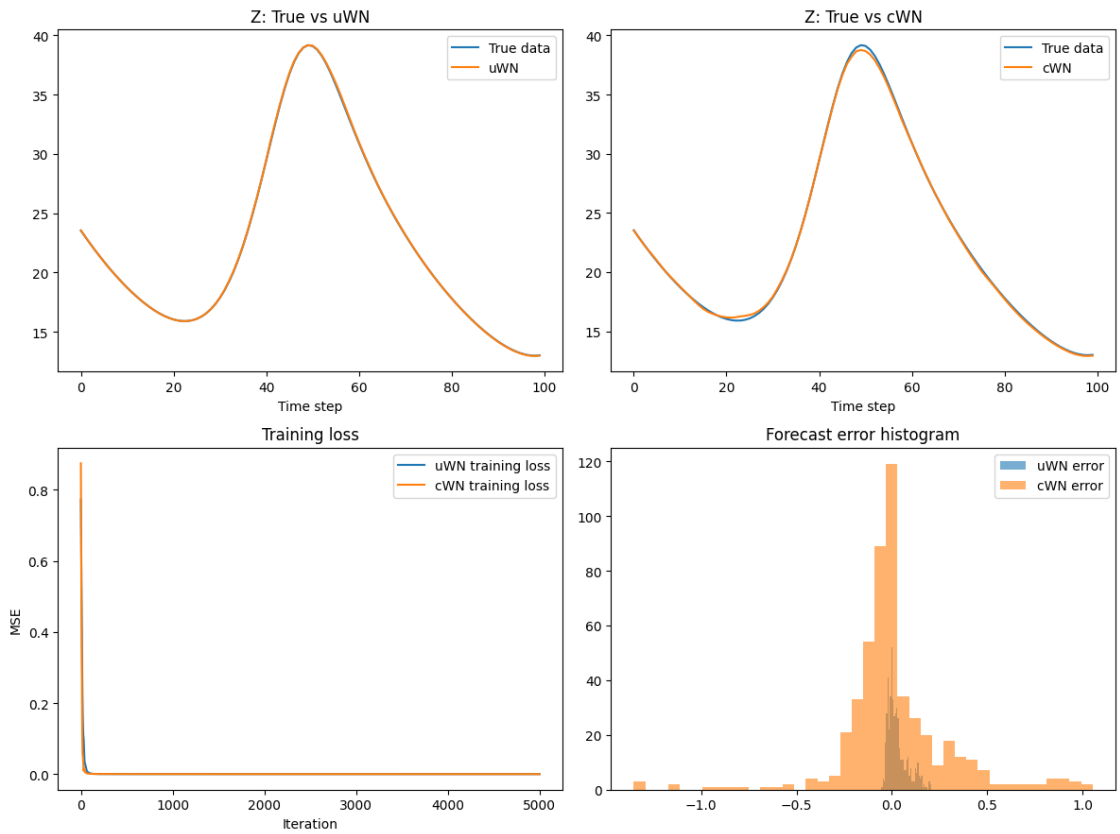
```
Using device: cuda
Figure save directory: d:\Columbia\AlgoTrading\FinalProject\IEOR4733-Final-
Project\results\lorenz_test

Running experiment for X…
X: RMSE uWN (std) = 0.018441, RMSE cWN (std) = 0.014334, RMSE uWN (orig) =
0.127202, RMSE cWN (orig) = 0.098871
```




```
Saved figure: d:\Columbia\AlgoTrading\FinalProject\IEOR4733-Final-
Project\results\lorenz_test\lorenz_X.png

Running experiment for Y…
Y: RMSE uWN (std) = 0.022610, RMSE cWN (std) = 0.032951, RMSE uWN (orig) =
0.184382, RMSE cWN (orig) = 0.268704

<Figure size 640x480 with 0 Axes>
```




```
Saved figure: d:\Columbia\AlgoTrading\FinalProject\IEOR4733-Final-
Project\results\lorenz_test\lorenz_Y.png

Running experiment for Z…
Z: RMSE uWN (std) = 0.006412, RMSE cWN (std) = 0.030777, RMSE uWN (orig) =
0.059576, RMSE cWN (orig) = 0.285978

<Figure size 640x480 with 0 Axes>
```




```
Saved figure: d:\Columbia\AlgoTrading\FinalProject\IEOR4733-Final-
Project\results\lorenz_test\lorenz_Z.png

Saved RMSE summary to: d:\Columbia\AlgoTrading\FinalProject\IEOR4733-Final-
Project\results\lorenz_test\rmse_summary.csv

===== RMSE Summary =====
Coordinate  RMSE_uWN_std  RMSE_cWN_std  RMSE_uWN_orig  RMSE_cWN_orig
         X      0.018441      0.014334       0.127202       0.098871
         Y      0.022610      0.032951       0.184382       0.268704
         Z      0.006412      0.030777       0.059576       0.285978

<Figure size 640x480 with 0 Axes>
```

The Lorenz test results provide an additional validation of the reproduced uWN and cWN model structures. The table reports the RMSE values for three coordinates, `x`, `y`, and `z`, under both the standardized setting and the original scale.

Overall, the RMSE values remain relatively small, especially for the standardized outputs. For the `x` coordinate, cWN achieves a lower standardized RMSE than uWN, with values of 0.014334 and 0.018441 respectively. This indicates that the conditional WaveNet is able to improve the fitting accuracy when market condition information is incorporated. For the original-scale results, cWN also improves upon uWN for the `x` coordinate, reducing the RMSE from 0.127202 to 0.098871.

Although cWN does not dominate uWN for every coordinate, the results are generally consistent with the expected behavior discussed in the paper. The conditional model can capture additional information through the conditioning variables, while the unconditional model still provides a stable benchmark. The Lorenz test therefore confirms that the reproduced model is able to generate reasonable and comparable outputs.

Based on these results, the model implementation is considered successful. The uWN and cWN structures have been correctly constructed and can be further integrated into the full training, validation, portfolio construction, and backtesting pipeline.

0.4 3. Backtesting Pipeline

In this section, we run the main backtesting pipeline for the uWN, cWN, and LSTM models. The corresponding implementation is provided in `./model/backtest_uWNcWN_LSTM2.py`.

This module integrates the major components of the empirical experiment, including feature loading, sample construction, rolling-window training, validation, prediction generation, portfolio construction, performance evaluation, and result visualization. The notebook imports and reloads the backtesting module to ensure that the latest version of the Python script is used.

The backtest is executed by calling `backtest_uWNcWN_LSTM2.backtest()`. The argument `--results_dir` is specified so that all generated outputs are saved into `./results/uWNcWN_LSTM2/`. These outputs include performance metrics, daily returns, cumulative returns, model predictions, and backtest plots.

This section serves as the final execution stage of the pipeline. After the data has been prepared and the model structures have been reproduced and validated, the backtesting module evaluates the out-of-sample performance of the models and compares their portfolio-level results.

[ ]:
```
from model import backtest_uWNcWN_LSTM2

importlib.reload(backtest_uWNcWN_LSTM2)
```

[ ]:
```
<module 'model.backtest_uWNcWN_LSTM2' from
'd:\\Columbia\\AlgoTrading\\FinalProject\\IEOR4733-Final-
Project\\model\\backtest_uWNcWN_LSTM2.py'>
```

[ ]:
```
backtest_uWNcWN_LSTM2.backtest([
    "--results_dir", "results/uWNcWN_LSTM2"
])
```

```
Using device: cuda

[1/4] Loading OHLCV …
[2/4] Loading market signals …
[3/4] Computing stock features …
  Computing features for 503 tickers …
    … 0/503
    … 100/503
    … 200/503
    … 300/503
```

```
    … 400/503
    … 500/503
[4/4] Computing market signals …
Merging & cleaning …
  Dataset: 1,776,305 rows | 3770 dates | 501 tickers

  Fold 1  test: 2013-07-08 → 2013-10-03
  Train: 205,190 samples | Val: 42,787 samples
  X_tr_n finite: True | max abs: 255.6279
  X_vl_n finite: True | max abs: 76.5551
  uWN  - trained 21 epochs | best val loss: 0.665826
  cWN  - trained 21 epochs | best val loss: 0.655025
  LSTM - trained 21 epochs | best val loss: 0.628354

  Fold 2  test: 2013-10-04 → 2014-01-03
  Train: 206,172 samples | Val: 43,059 samples
  X_tr_n finite: True | max abs: 161.6718
  X_vl_n finite: True | max abs: 193.0156
  uWN  - trained 21 epochs | best val loss: 0.553998
  cWN  - trained 21 epochs | best val loss: 0.564623
  LSTM - trained 22 epochs | best val loss: 0.546053

  Fold 3  test: 2014-01-06 → 2014-04-04
  Train: 207,150 samples | Val: 43,302 samples
  X_tr_n finite: True | max abs: 166.2470
  X_vl_n finite: True | max abs: 1075005685760.0000
  uWN  - trained 21 epochs | best val loss: 1500255833030656.000000
  cWN  - trained 21 epochs | best val loss: 839192049352704.000000
  LSTM - trained 21 epochs | best val loss: 0.562062

  Fold 4  test: 2014-04-07 → 2014-07-07
  Train: 208,120 samples | Val: 43,515 samples
  X_tr_n finite: True | max abs: 187.3585
  X_vl_n finite: True | max abs: 44448218087424.0000
  uWN  - trained 21 epochs | best val loss: 1573878052516003840.000000
  cWN  - trained 30 epochs | best val loss: 1101998997955936256.000000
  LSTM - trained 21 epochs | best val loss: 0.789837

  Fold 5  test: 2014-07-08 → 2014-10-03
  Train: 209,120 samples | Val: 43,729 samples
  X_tr_n finite: True | max abs: 669.4127
  X_vl_n finite: True | max abs: 29320.6230
  uWN  - trained 62 epochs | best val loss: 2.075557
  cWN  - trained 47 epochs | best val loss: 1.107439
  LSTM - trained 21 epochs | best val loss: 0.800794

  Fold 6  test: 2014-10-06 → 2015-01-05
  Train: 210,076 samples | Val: 43,953 samples
```

```
X_tr_n finite: True | max abs: 466.0492
X_vl_n finite: True | max abs: 587.5872
uWN  - trained 21 epochs | best val loss: 0.637371
cWN  - trained 21 epochs | best val loss: 0.685977
LSTM - trained 21 epochs | best val loss: 0.622785

Fold 7  test: 2015-01-06 → 2015-04-07
Train: 210,963 samples | Val: 44,138 samples
X_tr_n finite: True | max abs: 338.0607
X_vl_n finite: True | max abs: 426.2218
uWN  - trained 40 epochs | best val loss: 1.038754
cWN  - trained 36 epochs | best val loss: 1.102376
LSTM - trained 21 epochs | best val loss: 1.011428

Fold 8  test: 2015-04-08 → 2015-07-07
Train: 211,977 samples | Val: 44,250 samples
X_tr_n finite: True | max abs: 327.4770
X_vl_n finite: True | max abs: 35.1326
uWN  - trained 21 epochs | best val loss: 1.177761
cWN  - trained 77 epochs | best val loss: 1.284697
LSTM - trained 21 epochs | best val loss: 1.155557

Fold 9  test: 2015-07-08 → 2015-10-05
Train: 212,966 samples | Val: 44,345 samples
X_tr_n finite: True | max abs: 303.3175
X_vl_n finite: True | max abs: 189.6831
uWN  - trained 21 epochs | best val loss: 0.857112
cWN  - trained 21 epochs | best val loss: 0.910932
LSTM - trained 21 epochs | best val loss: 0.846702

Fold 10  test: 2015-10-06 → 2016-01-05
Train: 213,985 samples | Val: 44,480 samples
X_tr_n finite: True | max abs: 304.0423
X_vl_n finite: True | max abs: 276.1153
uWN  - trained 23 epochs | best val loss: 1.510142
cWN  - trained 21 epochs | best val loss: 1.549793
LSTM - trained 22 epochs | best val loss: 1.407972

Fold 11  test: 2016-01-06 → 2016-04-06
Train: 214,885 samples | Val: 44,669 samples
X_tr_n finite: True | max abs: 281.8040
X_vl_n finite: True | max abs: 255.9196
uWN  - trained 21 epochs | best val loss: 2.042357
cWN  - trained 31 epochs | best val loss: 2.099276
LSTM - trained 21 epochs | best val loss: 1.865587

Fold 12  test: 2016-04-07 → 2016-07-06
Train: 215,725 samples | Val: 44,804 samples
```

```
X_tr_n finite: True | max abs: 256.9782
X_vl_n finite: True | max abs: 247.3404
uWN  - trained 21 epochs | best val loss: 1.924471
cWN  - trained 21 epochs | best val loss: 1.992258
LSTM - trained 21 epochs | best val loss: 1.871171

Fold 13  test: 2016-07-07 → 2016-10-04
Train: 216,561 samples | Val: 44,853 samples
X_tr_n finite: True | max abs: 243.9206
X_vl_n finite: True | max abs: 234.7726
uWN  - trained 22 epochs | best val loss: 1.180996
cWN  - trained 21 epochs | best val loss: 1.153470
LSTM - trained 25 epochs | best val loss: 1.159373

Fold 14  test: 2016-10-05 → 2017-01-04
Train: 217,283 samples | Val: 45,006 samples
X_tr_n finite: True | max abs: 250.6512
X_vl_n finite: True | max abs: 225.9675
uWN  - trained 21 epochs | best val loss: 0.903766
cWN  - trained 21 epochs | best val loss: 0.915967
LSTM - trained 21 epochs | best val loss: 0.867332

Fold 15  test: 2017-01-05 → 2017-04-05
Train: 217,945 samples | Val: 45,218 samples
X_tr_n finite: True | max abs: 227.3089
X_vl_n finite: True | max abs: 184.6444
uWN  - trained 23 epochs | best val loss: 0.824560
cWN  - trained 21 epochs | best val loss: 0.823691
LSTM - trained 21 epochs | best val loss: 0.806400

Fold 16  test: 2017-04-06 → 2017-07-06
Train: 218,544 samples | Val: 45,355 samples
X_tr_n finite: True | max abs: 239.6078
X_vl_n finite: True | max abs: 200.4542
uWN  - trained 21 epochs | best val loss: 0.632541
cWN  - trained 21 epochs | best val loss: 0.646816
LSTM - trained 21 epochs | best val loss: 0.625550

Fold 17  test: 2017-07-07 → 2017-10-04
Train: 219,217 samples | Val: 45,396 samples
X_tr_n finite: True | max abs: 233.5117
X_vl_n finite: True | max abs: 195.3542
uWN  - trained 21 epochs | best val loss: 0.582268
cWN  - trained 21 epochs | best val loss: 0.590310
LSTM - trained 21 epochs | best val loss: 0.575492

Fold 18  test: 2017-10-05 → 2018-01-04
Train: 219,910 samples | Val: 45,437 samples
```

```
X_tr_n finite: True | max abs: 226.0707
X_vl_n finite: True | max abs: 227.2336
uWN  - trained 21 epochs | best val loss: 0.590774
cWN  - trained 21 epochs | best val loss: 0.613201
LSTM - trained 21 epochs | best val loss: 0.588895

Fold 19  test: 2018-01-05 → 2018-04-06
Train: 220,537 samples | Val: 45,654 samples
X_tr_n finite: True | max abs: 226.5195
X_vl_n finite: True | max abs: 227.6848
uWN  - trained 22 epochs | best val loss: 0.615094
cWN  - trained 21 epochs | best val loss: 0.620602
LSTM - trained 21 epochs | best val loss: 0.614118

Fold 20  test: 2018-04-09 → 2018-07-06
Train: 221,091 samples | Val: 45,878 samples
X_tr_n finite: True | max abs: 234.7629
X_vl_n finite: True | max abs: 194.4919
uWN  - trained 21 epochs | best val loss: 1.205957
cWN  - trained 21 epochs | best val loss: 1.472930
LSTM - trained 21 epochs | best val loss: 1.136728

Fold 21  test: 2018-07-09 → 2018-10-04
Train: 221,698 samples | Val: 45,988 samples
X_tr_n finite: True | max abs: 224.7375
X_vl_n finite: True | max abs: 186.1861
uWN  - trained 21 epochs | best val loss: 1.331790
cWN  - trained 21 epochs | best val loss: 1.284738
LSTM - trained 21 epochs | best val loss: 1.278561

Fold 22  test: 2018-10-05 → 2019-01-07
Train: 222,402 samples | Val: 46,132 samples
X_tr_n finite: True | max abs: 257.8318
X_vl_n finite: True | max abs: 263.2023
uWN  - trained 23 epochs | best val loss: 0.938407
cWN  - trained 21 epochs | best val loss: 0.946457
LSTM - trained 21 epochs | best val loss: 0.940395

Fold 23  test: 2019-01-08 → 2019-04-08
Train: 223,119 samples | Val: 46,172 samples
X_tr_n finite: True | max abs: 263.3304
X_vl_n finite: True | max abs: 268.8155
uWN  - trained 21 epochs | best val loss: 2.169581
cWN  - trained 21 epochs | best val loss: 2.177105
LSTM - trained 21 epochs | best val loss: 2.038013

Fold 24  test: 2019-04-09 → 2019-07-09
Train: 223,760 samples | Val: 46,209 samples
```

```
X_tr_n finite: True | max abs: 269.6321
X_vl_n finite: True | max abs: 233.0001
uWN  - trained 21 epochs | best val loss: 1.983925
cWN  - trained 23 epochs | best val loss: 2.063324
LSTM - trained 21 epochs | best val loss: 1.801575

Fold 25  test: 2019-07-10 → 2019-10-07
Train: 224,305 samples | Val: 46,269 samples
X_tr_n finite: True | max abs: 256.4307
X_vl_n finite: True | max abs: 221.5922
uWN  - trained 21 epochs | best val loss: 0.929249
cWN  - trained 22 epochs | best val loss: 0.954579
LSTM - trained 21 epochs | best val loss: 0.926672

Fold 26  test: 2019-10-08 → 2020-01-07
Train: 224,846 samples | Val: 46,285 samples
X_tr_n finite: True | max abs: 257.4043
X_vl_n finite: True | max abs: 119.9000
uWN  - trained 21 epochs | best val loss: 1.140134
cWN  - trained 21 epochs | best val loss: 1.136138
LSTM - trained 21 epochs | best val loss: 1.089758

Fold 27  test: 2020-01-08 → 2020-04-07
Train: 225,411 samples | Val: 46,348 samples
X_tr_n finite: True | max abs: 246.4873
X_vl_n finite: True | max abs: 101.7293
uWN  - trained 21 epochs | best val loss: 0.899527
cWN  - trained 21 epochs | best val loss: 0.902888
LSTM - trained 21 epochs | best val loss: 0.873160

Fold 28  test: 2020-04-08 → 2020-07-08
Train: 225,887 samples | Val: 46,420 samples
X_tr_n finite: True | max abs: 290.9834
X_vl_n finite: True | max abs: 348.5241
uWN  - trained 21 epochs | best val loss: 6.281889
cWN  - trained 21 epochs | best val loss: 6.154244
LSTM - trained 21 epochs | best val loss: 6.006588

Fold 29  test: 2020-07-09 → 2020-10-06
Train: 226,221 samples | Val: 46,632 samples
X_tr_n finite: True | max abs: 284.6111
X_vl_n finite: True | max abs: 391.9025
uWN  - trained 21 epochs | best val loss: 7.636308
cWN  - trained 22 epochs | best val loss: 8.395042
LSTM - trained 21 epochs | best val loss: 7.741488

Fold 30  test: 2020-10-07 → 2021-01-06
Train: 226,545 samples | Val: 46,947 samples
```

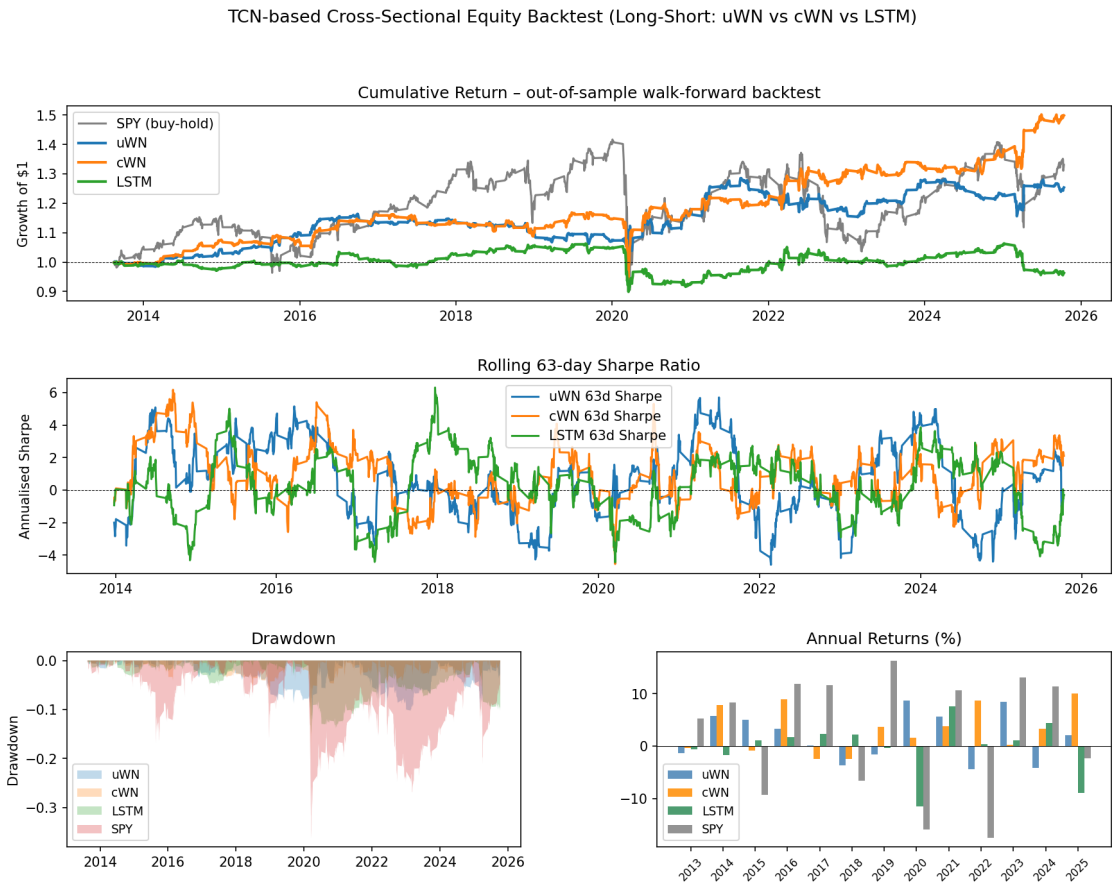
```
X_tr_n finite: True | max abs: 301.7771
X_vl_n finite: True | max abs: 346.9348
uWN  - trained 24 epochs | best val loss: 0.939693
cWN  - trained 26 epochs | best val loss: 0.944602
LSTM - trained 21 epochs | best val loss: 0.946168

Fold 31  test: 2021-01-07 → 2021-04-08
Train: 226,943 samples | Val: 47,092 samples
X_tr_n finite: True | max abs: 280.6060
X_vl_n finite: True | max abs: 241.4874
uWN  - trained 21 epochs | best val loss: 0.706575
cWN  - trained 21 epochs | best val loss: 0.720590
LSTM - trained 21 epochs | best val loss: 0.715054

Fold 32  test: 2021-04-09 → 2021-07-08
Train: 227,500 samples | Val: 47,142 samples
X_tr_n finite: True | max abs: 271.1015
X_vl_n finite: True | max abs: 37.4503
uWN  - trained 21 epochs | best val loss: 0.577372
cWN  - trained 21 epochs | best val loss: 0.577097
LSTM - trained 21 epochs | best val loss: 0.574708

Fold 33  test: 2021-07-09 → 2021-10-06
Train: 228,111 samples | Val: 47,238 samples
X_tr_n finite: True | max abs: 271.4653
X_vl_n finite: True | max abs: 37.6090
uWN  - trained 22 epochs | best val loss: 0.457266
cWN  - trained 21 epochs | best val loss: 0.466266
LSTM - trained 21 epochs | best val loss: 0.446646

Fold 34  test: 2021-10-07 → 2022-01-05
Train: 228,678 samples | Val: 47,336 samples
X_tr_n finite: True | max abs: 294.4973
X_vl_n finite: True | max abs: 55.9049
uWN  - trained 21 epochs | best val loss: 0.327032
cWN  - trained 21 epochs | best val loss: 0.340818
LSTM - trained 21 epochs | best val loss: 0.318438

Fold 35  test: 2022-01-06 → 2022-04-06
Train: 229,341 samples | Val: 47,375 samples
X_tr_n finite: True | max abs: 297.7372
X_vl_n finite: True | max abs: 56.5201
uWN  - trained 21 epochs | best val loss: 0.419857
cWN  - trained 21 epochs | best val loss: 0.423943
LSTM - trained 21 epochs | best val loss: 0.425753

Fold 36  test: 2022-04-07 → 2022-07-08
Train: 229,989 samples | Val: 47,549 samples
```

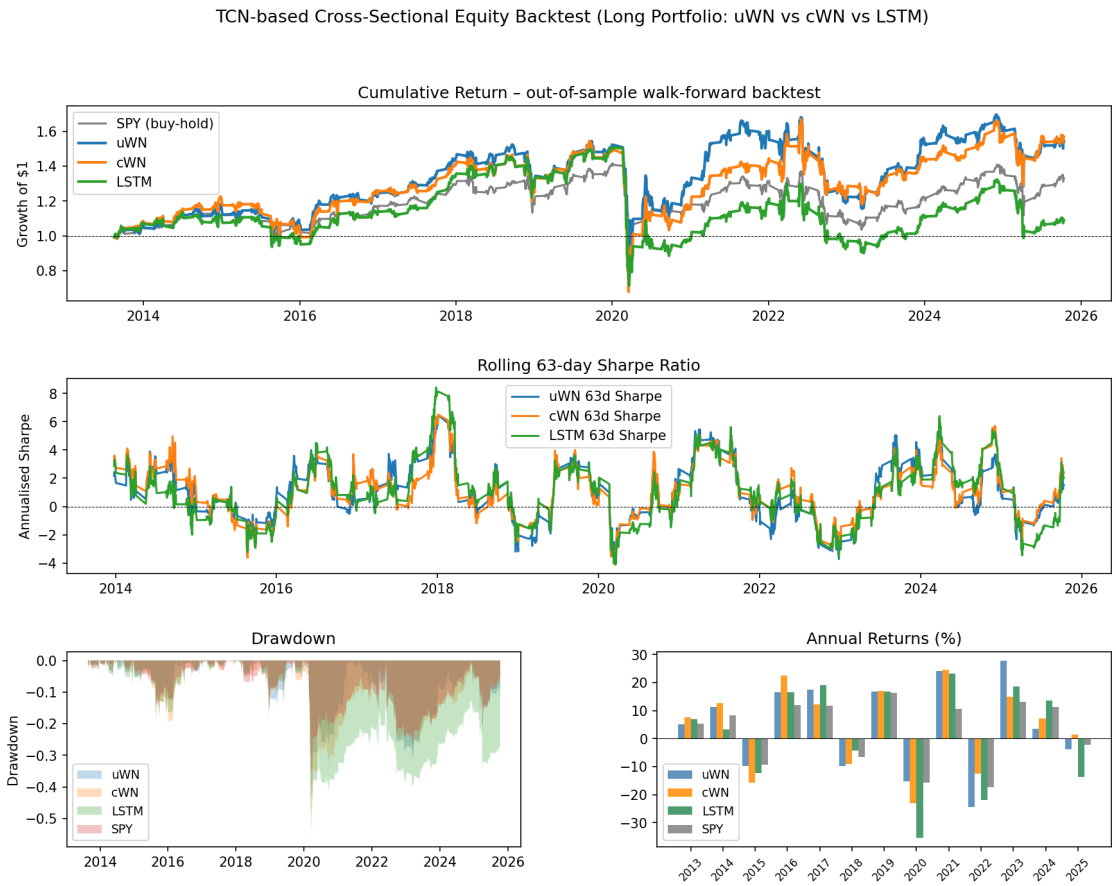
```
X_tr_n finite: True | max abs: 302.7461
X_vl_n finite: True | max abs: 31.5502
uWN  - trained 21 epochs | best val loss: 0.708543
cWN  - trained 22 epochs | best val loss: 0.743535
LSTM - trained 21 epochs | best val loss: 0.711676

Fold 37  test: 2022-07-11 → 2022-10-06
Train: 230,638 samples | Val: 47,762 samples
X_tr_n finite: True | max abs: 301.7318
X_vl_n finite: True | max abs: 31.7930
uWN  - trained 21 epochs | best val loss: 0.974192
cWN  - trained 21 epochs | best val loss: 0.984729
LSTM - trained 21 epochs | best val loss: 0.975392

Fold 38  test: 2022-10-07 → 2023-01-06
Train: 231,331 samples | Val: 47,931 samples
X_tr_n finite: True | max abs: 486.6801
X_vl_n finite: True | max abs: 34.1669
uWN  - trained 21 epochs | best val loss: 1.424441
cWN  - trained 21 epochs | best val loss: 1.471098
LSTM - trained 21 epochs | best val loss: 1.328148

Fold 39  test: 2023-01-09 → 2023-04-10
Train: 231,931 samples | Val: 48,001 samples
X_tr_n finite: True | max abs: 503.3731
X_vl_n finite: True | max abs: 37.3144
uWN  - trained 21 epochs | best val loss: 1.382588
cWN  - trained 21 epochs | best val loss: 1.426573
LSTM - trained 21 epochs | best val loss: 1.286484

Fold 40  test: 2023-04-11 → 2023-07-11
Train: 232,505 samples | Val: 48,041 samples
X_tr_n finite: True | max abs: 482.1867
X_vl_n finite: True | max abs: 38.0670
uWN  - trained 21 epochs | best val loss: 0.942693
cWN  - trained 21 epochs | best val loss: 0.979329
LSTM - trained 21 epochs | best val loss: 0.926600

Fold 41  test: 2023-07-12 → 2023-10-09
Train: 233,072 samples | Val: 48,104 samples
X_tr_n finite: True | max abs: 482.7743
X_vl_n finite: True | max abs: 2572.9475
uWN  - trained 21 epochs | best val loss: 0.815257
cWN  - trained 21 epochs | best val loss: 0.860412
LSTM - trained 21 epochs | best val loss: 0.764612

Fold 42  test: 2023-10-10 → 2024-01-09
Train: 233,637 samples | Val: 48,112 samples
```

```
X_tr_n finite: True | max abs: 483.3590
X_vl_n finite: True | max abs: 2576.0642
uWN  - trained 21 epochs | best val loss: 0.675758
cWN  - trained 21 epochs | best val loss: 0.679769
LSTM - trained 21 epochs | best val loss: 0.659760

Fold 43  test: 2024-01-10 → 2024-04-10
Train: 234,141 samples | Val: 48,112 samples
X_tr_n finite: True | max abs: 475.5807
X_vl_n finite: True | max abs: 158.3952
uWN  - trained 21 epochs | best val loss: 0.674710
cWN  - trained 21 epochs | best val loss: 0.690914
LSTM - trained 21 epochs | best val loss: 0.648509

Fold 44  test: 2024-04-11 → 2024-07-11
Train: 234,640 samples | Val: 48,161 samples
X_tr_n finite: True | max abs: 477.3861
X_vl_n finite: True | max abs: 44.0844
uWN  - trained 22 epochs | best val loss: 0.652533
cWN  - trained 21 epochs | best val loss: 0.651440
LSTM - trained 21 epochs | best val loss: 0.652185

Fold 45  test: 2024-07-12 → 2024-10-09
Train: 235,044 samples | Val: 48,226 samples
X_tr_n finite: True | max abs: 459.9710
X_vl_n finite: True | max abs: 75.1837
uWN  - trained 21 epochs | best val loss: 0.658595
cWN  - trained 21 epochs | best val loss: 0.672537
LSTM - trained 25 epochs | best val loss: 0.654621

Fold 46  test: 2024-10-10 → 2025-01-10
Train: 235,368 samples | Val: 48,289 samples
X_tr_n finite: True | max abs: 460.2879
X_vl_n finite: True | max abs: 78.9172
uWN  - trained 21 epochs | best val loss: 0.887537
cWN  - trained 21 epochs | best val loss: 0.957297
LSTM - trained 21 epochs | best val loss: 0.846336

Fold 47  test: 2025-01-13 → 2025-04-11
Train: 235,581 samples | Val: 48,344 samples
X_tr_n finite: True | max abs: 460.4962
X_vl_n finite: True | max abs: 32.9370
uWN  - trained 21 epochs | best val loss: 0.977660
cWN  - trained 21 epochs | best val loss: 1.132861
LSTM - trained 21 epochs | best val loss: 0.927907

Fold 48  test: 2025-04-14 → 2025-07-15
Train: 235,784 samples | Val: 48,403 samples
```

```
  X_tr_n finite: True | max abs: 460.6945
  X_vl_n finite: True | max abs: 31.4100
  uWN  - trained 21 epochs | best val loss: 1.798038
  cWN  - trained 35 epochs | best val loss: 1.703831
  LSTM - trained 21 epochs | best val loss: 1.810081

  Fold 49  test: 2025-07-16 → 2025-10-13
  Train: 236,011 samples | Val: 48,492 samples
  X_tr_n finite: True | max abs: 460.9162
  X_vl_n finite: True | max abs: 38.6841
  uWN  - trained 21 epochs | best val loss: 1.818112
  cWN  - trained 21 epochs | best val loss: 1.890923
  LSTM - trained 24 epochs | best val loss: 1.757000

======================================================================
  BACKTEST RESULTS - OUT-OF-SAMPLE
======================================================================
Strategy            CumRet%  AnnRet%  AnnVol%  Sharpe   MaxDD%  HitRate%
----------------------------------------------------------------------
uWN (L/S)              25.4      3.5      7.3   0.476    -10.8      51.4
cWN (L/S)              49.9      6.3      8.7   0.726    -21.0      51.3
LSTM (L/S)             -3.7     -0.6      6.7  -0.085    -15.4      50.6
uWN (Long)             54.4      6.8     24.2   0.280    -43.9      54.8
cWN (Long)             56.9      7.1     25.6   0.275    -56.1      54.5
LSTM (Long)             9.1      1.3     24.1   0.055    -52.7      54.6
SPY buy-hold           33.1      4.4     18.9   0.233    -36.2      53.7
----------------------------------------------------------------------
Plot saved → results\uWNcWN_LSTM2\backtest_ls_without_lstm.png
Plot saved → results\uWNcWN_LSTM2\backtest_ls_with_lstm.png
Plot saved → results\uWNcWN_LSTM2\backtest_long_with_lstm.png
```

[ ]:
```python
from IPython.display import Image, display

display(Image(filename="results/uWNcWN_LSTM2/backtest_ls_with_lstm.png"))
display(Image(filename="results/uWNcWN_LSTM2/backtest_long_with_lstm.png"))
```


TCN-based Cross-Sectional Equity Backtest (Long-Short: uWN vs cWN vs LSTM)
Cumulative Return – out-of-sample walk-forward backtest
Growth of $1
SPY (buy-hold)
uWN
cWN
LSTM
Rolling 63-day Sharpe Ratio
Annualised Sharpe
uWN 63d Sharpe
cWN 63d Sharpe
LSTM 63d Sharpe
Drawdown
uWN
cWN
LSTM
SPY
Annual Returns (%)



The backtesting results show clear differences across the three models under both long-short and long-only portfolio settings.

From the long-short cumulative return plot, cWN achieves the strongest overall performance over the full out-of-sample period. Its equity curve grows more steadily after 2020 and ends at the highest level among the three models, reaching close to 1.5 times the initial capital. uWN also generates positive cumulative returns, but its final performance is lower than cWN and its growth becomes flatter after 2022. In contrast, the LSTM strategy remains close to or below the initial capital for most of the sample period, suggesting weaker predictive power in this cross-sectional setting.

The rolling 63-day Sharpe ratio further supports this observation. Both uWN and cWN experience periods of positive and negative risk-adjusted performance, but cWN shows stronger positive Sharpe episodes, especially near the end of the sample. The drawdown plot indicates that all strategies experience losses during stressed periods, particularly around 2020, but the model-based long-short strategies generally have smaller drawdowns than SPY. The annual return chart also shows that cWN produces more consistently positive yearly returns than uWN and LSTM in several years.

The long-only portfolio results provide an additional perspective. In the cumulative return plot, all three models are more closely related to the market trend because the portfolios only take long positions. Before 2020, uWN and cWN both achieve strong growth, with uWN slightly leading in

some periods. However, after the market disruption around 2020, cWN recovers more effectively and remains competitive through the later sample period. LSTM also improves under the long-only setting compared with its long-short result, but it still does not consistently outperform uWN or cWN. The rolling Sharpe ratios of the long-only strategies move closely together, showing that their risk-adjusted performance is more affected by common market conditions. The drawdown plot also shows larger downside exposure than the long-short case, especially during market stress, which is expected because the long-only portfolios cannot hedge by taking short positions.

The summary table provides numerical support for these graphical results. For the long-short portfolios, cWN achieves the highest cumulative return of 49.9%, the highest annual return of 6.3%, and the highest Sharpe ratio of 0.726. uWN also performs positively, with a cumulative return of 25.4% and a Sharpe ratio of 0.476. By comparison, LSTM has a negative cumulative return of -3.7% and a slightly negative Sharpe ratio of -0.085, indicating that it fails to generate stable out-of-sample alpha in the long-short setting.

For the long-only portfolios, all three models generate positive cumulative returns. cWN again achieves the highest cumulative return at 56.9%, slightly outperforming uWN at 54.4%. However, the long-only strategies also have much higher annualized volatility and larger maximum drawdowns than their long-short versions. This suggests that the long-only portfolios benefit from market exposure but are also more vulnerable to broad market downturns. Overall, the results suggest that the conditional WaveNet model benefits from incorporating market condition information and provides the best overall out-of-sample performance among the tested models, especially in the long-short portfolio setting.

0.5 4. Regime Analysis

After the main backtest is completed, I added a regime analysis module to examine whether the model-based strategies perform differently across market environments. The implementation is stored in `./robustness/regime_analysis.py`, and the Streamlit visualization version is stored in `./pages/03_regime_analysis.py`.

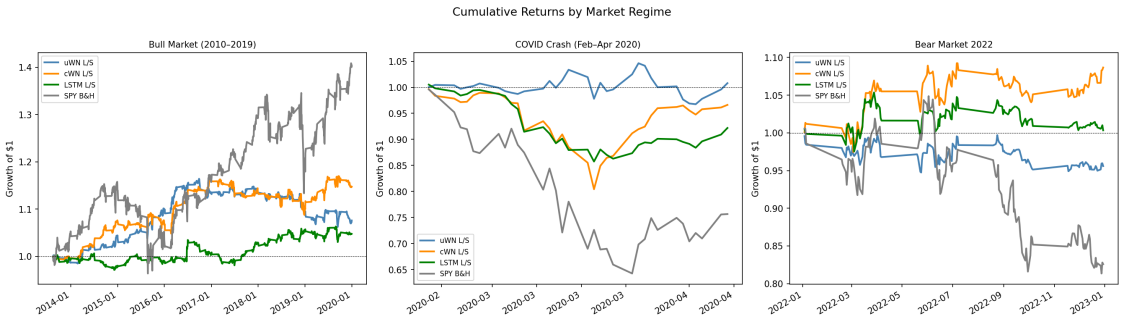
The regime analysis uses the daily return file generated by the main backtest `./results/uWNcWN_LSTM2/daily_returns.csv`. It then evaluates the uWN, cWN, LSTM, and SPY benchmark returns over several representative periods, including the bull market before 2020, the COVID crash, and the 2022 bear market. For each regime, the script computes performance statistics such as annualized return, annualized volatility, Sharpe ratio, and maximum drawdown.

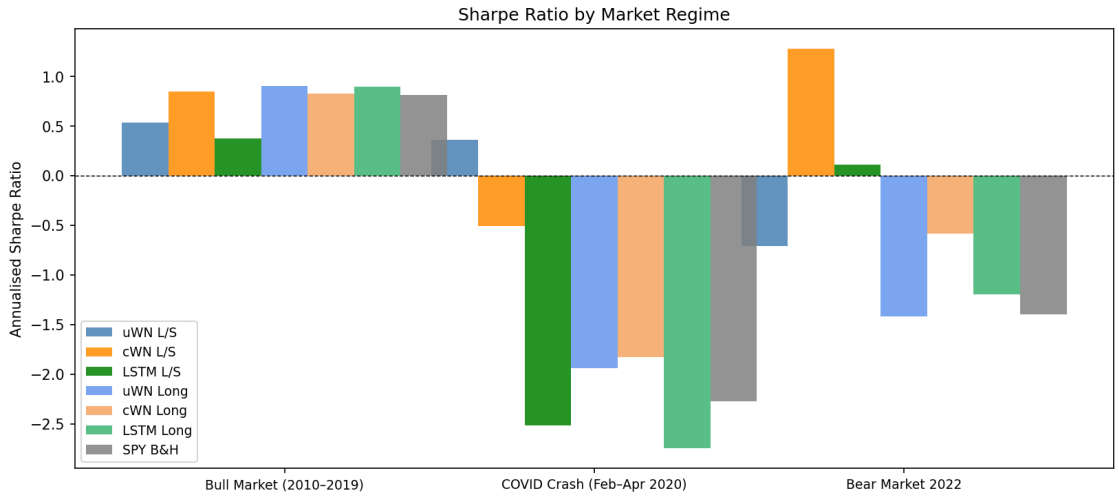
[ ]:
```
# Run the regime analysis module after the main backtest has generated␣
↪daily_returns.csv.
# This command saves regime-level tables and plots into the selected results␣
↪folder.

!python robustness/regime_analysis.py --results_dir results/uWNcWN_LSTM2
```

```
======================================================================
  REGIME PERFORMANCE ANALYSIS
======================================================================
```

```
Bull Market (2010-2019)  [2013-08-16 → 2019-12-31]  (880 trading days)

Strategy              AnnRet%  AnnVol%   Sharpe   MaxDD%   Calmar    Hit%
---------------------------------------------------------------------------
uWN L/S                   2.2      4.0    0.538     -8.8     0.25    51.7
cWN L/S                   4.0      4.8    0.849     -6.6     0.61    52.4
LSTM L/S                  1.4      3.7    0.375     -5.2     0.27    49.4
uWN Long                 13.1     14.6    0.902    -17.8     0.74    56.1
cWN Long                 12.7     15.3    0.830    -19.1     0.66    55.7
LSTM Long                12.8     14.3    0.898    -17.6     0.73    56.1
SPY B&H                  10.6     12.9    0.816    -16.8     0.63    53.9


COVID Crash (Feb-Apr 2020)  [2020-02-20 → 2020-04-07]  (34 trading days)

Strategy              AnnRet%  AnnVol%   Sharpe   MaxDD%   Calmar    Hit%
---------------------------------------------------------------------------
uWN L/S                   8.8     24.4    0.362     -7.5     1.17    52.9
cWN L/S                 -18.7     37.0   -0.505    -19.3    -0.97    52.9
LSTM L/S                -57.6     22.9   -2.517    -14.7    -3.92    44.1
uWN Long               -193.1     99.5   -1.940    -42.6    -4.53    41.2
cWN Long               -217.6    118.8   -1.832    -53.9    -4.04    44.1
LSTM Long              -291.6    106.1   -2.749    -52.4    -5.56    41.2
SPY B&H                -176.4     77.5   -2.276    -35.5    -4.97    38.2


Bear Market 2022  [2022-01-03 → 2022-12-30]  (135 trading days)

Strategy              AnnRet%  AnnVol%   Sharpe   MaxDD%   Calmar    Hit%
---------------------------------------------------------------------------
uWN L/S                  -7.8     11.0   -0.708     -5.1    -1.55    43.7
cWN L/S                  16.3     12.8    1.279     -4.8     3.39    51.1
LSTM L/S                  1.3     11.6    0.112     -5.7     0.23    45.2
uWN Long                 -46.6     32.8   -1.420    -29.4    -1.59    45.2
cWN Long                 -19.5     33.2   -0.587    -27.4    -0.71    43.7
LSTM Long                -40.6     33.9   -1.198    -28.8    -1.41    45.2
SPY B&H                  -32.9     23.6   -1.397    -22.4    -1.47    41.5
Saved → results/uWNcWN_LSTM2/regime_Bull_Market_2010-2019.csv
Saved → results/uWNcWN_LSTM2/regime_COVID_Crash_Feb-Apr_2020.csv
Saved → results/uWNcWN_LSTM2/regime_Bear_Market_2022.csv
Regime plot saved → results/uWNcWN_LSTM2/regime_cumulative.png
Sharpe bar plot saved → results/uWNcWN_LSTM2/regime_sharpe.png
```

[ ]:
```
display(Image(filename="results/uWNcWN_LSTM2/regime_cumulative.png"))
display(Image(filename="results/uWNcWN_LSTM2/regime_sharpe.png"))
```





During the bull market period, all strategies generated relatively stable performance. The long-only portfolios performed especially well because they benefited from the upward market trend. The uWN Long, cWN Long, and LSTM Long portfolios all achieved Sharpe ratios close to or above the SPY benchmark, with uWN Long producing the highest Sharpe ratio of 0.902. Among the long-short strategies, cWN performed the best, with an annualized return of 4.0% and a Sharpe ratio of 0.849. This suggests that conditioning on market-level signals improved the model's ability to form relative-value portfolios during normal expansionary market conditions.

The COVID crash period shows a very different pattern. The long-only strategies suffered large losses because they remained exposed to the broad market decline. SPY also experienced a large negative annualized return and a Sharpe ratio of -2.276. In contrast, the uWN long-short strategy was much more resilient, producing a positive Sharpe ratio of 0.362 and a much smaller maximum drawdown of -7.5%. This indicates that the market-neutral long-short construction helped reduce downside exposure during the crash. However, cWN and LSTM long-short portfolios performed worse in this period, suggesting that the additional model structure did not necessarily improve robustness during extreme market stress.

In the 2022 bear market, the advantage of the cWN long-short strategy becomes much clearer. While SPY and all long-only strategies had negative annualized returns, cWN L/S achieved a

positive annualized return of 16.3%, a Sharpe ratio of 1.279, and the strongest Calmar ratio of 3.39. This result suggests that the conditional model was especially effective in a prolonged bear-market environment, where market signals may have contained useful information for ranking stocks. By contrast, uWN L/S had a negative Sharpe ratio of -0.708, while LSTM L/S was only slightly positive with a Sharpe ratio of 0.112.

Overall, the regime analysis shows that no single model dominates in every environment, but the cWN long-short strategy appears to be the most robust across regimes. It performs strongly in the bull market, delivers the best performance in the 2022 bear market, and remains competitive in risk-adjusted terms across the full set of regimes. The results also highlight the importance of portfolio construction: long-only strategies perform well in rising markets but become highly exposed during downturns, while long-short strategies can reduce market exposure and potentially generate returns from cross-sectional stock ranking. This supports the main conclusion that conditional forecasting signals are most valuable when they are combined with a portfolio construction method that controls market exposure.

The regime analysis outputs can then be viewed directly in the Streamlit dashboard through the `Regime Analysis` page. This page loads the same `daily_returns.csv` file, computes regime-specific Sharpe ratios, and visualizes cumulative returns across the selected market periods. The purpose is to compare the models not only by full-sample performance, but also by how they behave in normal, crisis, and bear-market conditions.

0.6 5. Transaction Cost Modeling and Portfolio Sensitivity

I also added a separate transaction cost modeling module. The script is stored in `./transaction_cost/transactioncost.py`. Unlike the main backtest script, this module does not retrain uWN, cWN, or LSTM. Instead, it assumes that the prediction files have already been generated by the main backtest and saved in the results folder.

The expected input files are:

```
results/uWNcWN_LSTM2/predictions_uWN.csv
results/uWNcWN_LSTM2/predictions_cWN.csv
results/uWNcWN_LSTM2/predictions_LSTM.csv
```

The module loads these saved prediction files, ranks stocks by predicted return, forms long-short and long-only portfolios, and deducts transaction costs based on turnover. The transaction cost is modeled as a one-way cost in basis points and is charged only on rebalance days. The module also allows the holding period to be changed, so the strategy can be tested under daily, weekly, biweekly, or monthly rebalancing assumptions.

This extension is important because the original backtest performance can be overstated if trading frictions are ignored. Since cross-sectional strategies may require frequent rebalancing, turnover and transaction costs can meaningfully reduce realized performance. The transaction cost module therefore provides a more realistic evaluation of whether the model predictions remain useful after implementation costs.

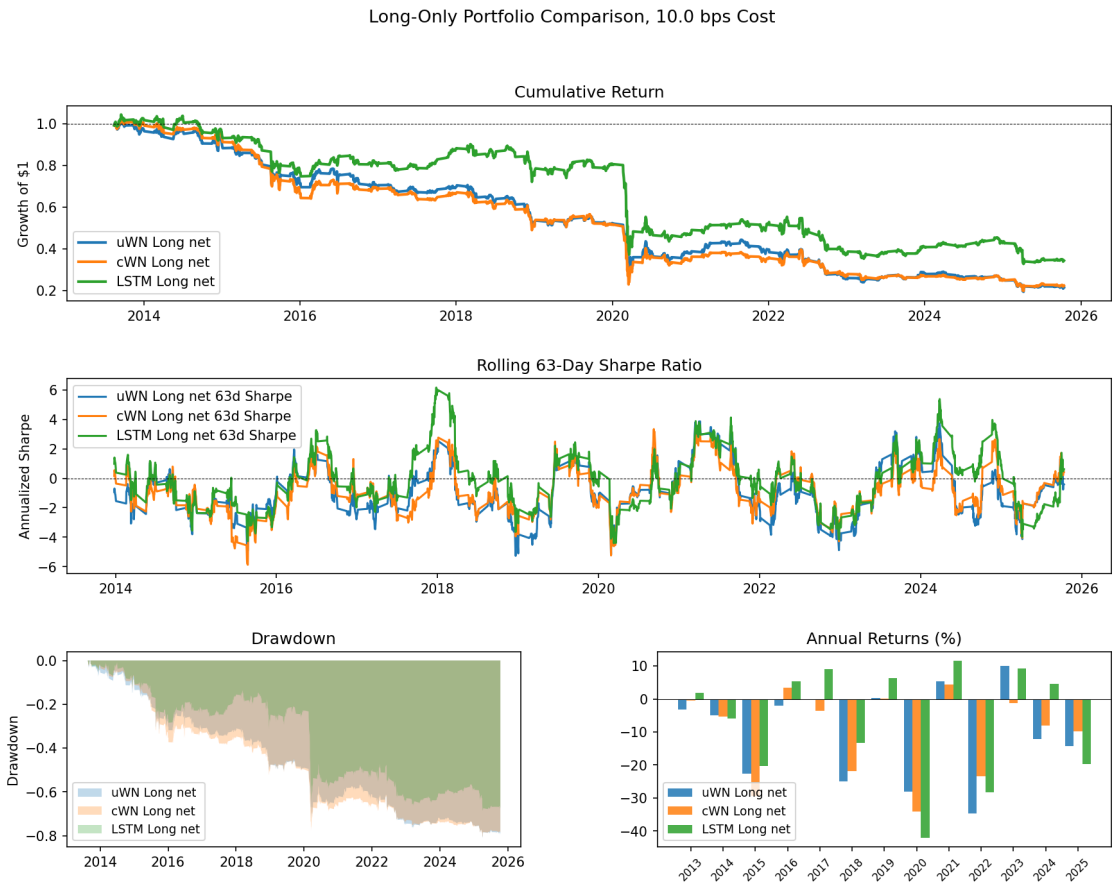
[ ]:
```
# Run transaction cost modeling using the saved prediction files from the main␣
↪backtest.
# The results are saved into the transaction_cost folder.
```

```
!python transaction_costs/transactioncost.py \
    --pred_dir results/uWNcWN_LSTM2 \
    --results_dir transaction_cost \
    --cost_bps 10 \
    --holding 1 \
    --top_frac 0.20 \
    --bot_frac 0.20
```

```
Loaded uWN: 796,267 rows from results/uWNcWN_LSTM2/predictions_uWN.csv
Loaded cWN: 796,267 rows from results/uWNcWN_LSTM2/predictions_cWN.csv
Loaded LSTM: 796,267 rows from results/uWNcWN_LSTM2/predictions_LSTM.csv
Formed portfolio for uWN: 1,666 days
Formed portfolio for cWN: 1,666 days
Formed portfolio for LSTM: 1,666 days

================================================================================
====================
PORTFOLIO PERFORMANCE COMPARISON
================================================================================
====================
           name  cum_return_pct  ann_return_pct  ann_vol_pct  sharpe
max_drawdown_pct  hit_rate_pct  n_days
    uWN L/S net          -83.19          -23.64         7.33  -3.224
-83.28         31.15    1666
  uWN L/S gross           25.37            3.48         7.31   0.476
-10.78         51.38    1666
   uWN Long net          -78.45          -20.72        24.24  -0.855
-80.91         49.04    1666
 uWN Long gross           54.37            6.79        24.21   0.280
-43.94         54.80    1666
    cWN L/S net          -79.44          -21.28         8.72  -2.440
-80.16         33.49    1666
  cWN L/S gross           49.88            6.31         8.70   0.726
-21.00         51.26    1666
   cWN Long net          -77.57          -20.24        25.63  -0.790
-80.61         48.32    1666
 cWN Long gross           56.92            7.05        25.61   0.275
-56.07         54.50    1666
   LSTM L/S net          -70.21          -16.74         6.69  -2.503
-70.27         38.66    1666
 LSTM L/S gross           -3.71           -0.57         6.67  -0.085
-15.44         50.60    1666
  LSTM Long net          -65.59          -14.90        24.08  -0.619
-68.55         51.26    1666
LSTM Long gross            9.05            1.32        24.06   0.055
-52.65         54.56    1666
================================================================================
====================
```

```
Saved plot: transaction_cost/long_short_comparison.png
Figure(1400x1000)
Saved plot: transaction_cost/long_only_comparison.png
Figure(1400x1000)

Saved outputs to: transaction_cost
```

[ ]:
```python
from IPython.display import Image, display

# Display transaction-cost-adjusted portfolio comparison plots.
display(Image(filename="transaction_cost/long_short_comparison.png"))
display(Image(filename="transaction_cost/long_only_comparison.png"))
```



The transaction cost module saves the following outputs:

```
transaction_cost/
  metrics.csv
  daily_returns.csv
  cumulative_returns.csv
  long_short_comparison.png
  long_only_comparison.png
```

The `metrics.csv` file compares the net long-short portfolio, gross long-short portfolio, and long-only portfolio for each model. The `daily_returns.csv` file stores strategy returns, turnover, and transaction cost information. The two plot files visualize the long-short and long-only cumulative performance separately.

The main interpretation from this extension is that portfolio performance should be evaluated under realistic trading assumptions. A strategy with strong gross returns may become much weaker once turnover costs are deducted. Therefore, this module allows the project to compare models not only by predictive accuracy or raw backtest returns, but also by implementable portfolio performance.

0.6.1 Transaction Cost Result Interpretation

In this experiment, I use a daily rebalancing strategy with a 20% long-short rule. Each day, the strategy goes long the top 20% of stocks ranked by predicted return and short the bottom 20%. I assume a one-way transaction cost of 10 basis points.

The results show that transaction costs have a very large effect on portfolio performance. After deducting transaction costs, all long-short strategies perform very poorly. The uWN L/S net portfolio has a cumulative return of -83.19% and a Sharpe ratio of -3.224. The cWN L/S net portfolio has a cumulative return of -79.44% and a Sharpe ratio of -2.440. The LSTM L/S net portfolio also declines substantially. The cumulative return plot shows a steady decline in all three net long-short strategies, indicating that daily turnover costs overwhelm the predictive signal. The long-only strategy is also not immune to transaction costs. Cumulative returns of uWN Long net falls to -78.45% and cWN Long net falls to -77.57%. LSTM Long net also becomes negative, with a cumulative return of -65.59%.

Overall, the transaction cost analysis shows that the original strategy is highly sensitive to implementation assumptions. The models can generate useful gross signals, especially for cWN, but daily rebalancing with a 10 bps one-way transaction cost makes both the long-short and long-only net strategies unattractive. So it is better to increase the holding period, and some future work can be applying a stronger signal threshold before trading, or using portfolio optimization methods that explicitly penalize turnover.

0.7 6. Streamlit Web App

To make the project easier to explore, I added a Streamlit web application. The entry point is `app.py`, and the individual dashboard pages are stored in the `./pages/` folder.

The web app contains four main pages:

1. `01_data_explorer.py`: displays dataset summary information, feature distributions, and ticker-level feature time series.
2. `02_backtest_results.py`: visualizes cumulative returns, rolling Sharpe ratios, drawdowns, annual returns, and performance metrics from the main backtest.
3. `03_regime_analysis.py`: shows strategy performance across different market regimes.
4. `04_run_simulation.py`: allows the user to interactively change portfolio rules, including long group size, short group size, holding period, transaction cost, and strategy type. This page uses pre-computed model predictions.

The app uses `st.session_state["results_dir"]` to pass the selected results folder across pages. This keeps the dashboard flexible because the user can switch between different result folders from the sidebar and immediately view the corresponding analysis.

[ ]:
```
# Launch the Streamlit dashboard from the project root.
# Run this command in the terminal, not inside a normal Python script.

!streamlit run app.py
```

The Streamlit app turns the project from a static backtesting pipeline into an interactive research dashboard.

The simulation page is especially useful for sensitivity analysis. It allows the user to test different

transaction costs, holding periods, and portfolio construction rules directly on the saved prediction files. This makes the project more practical because it separates model training from portfolio implementation.